CS 147: Lab 03
Due Date: October 8, 2026

September 3, 2026

The main objective of this problem set is to design a register file using the toolchain for the course project. Feel free to refer to the Verilog cheatsheet, Verilog lecture slides linked off of canvas.

Some advice: Before starting to write any Verilog, you should do the following:

1. Break down your design into sub-modules.
2. Define interfaces between these modules.
3. Draw paper and pencil schematics for these modules (these will be handed in as scanned `schematic.pdf` file(s)).
4. Then start writing Verilog.

Problem 1 (30 points)

Using Verilog, design an 8-by-16-bit register file (i.e., 8 registers, each of size 16 bits). See the Verilog interface below. It has one write port, two read ports, three register select inputs (two for read and one for write,) a write enable, a reset and a clock input. All register state changes occur on the rising edge of the clock. Your basic building block must be the **D-flipflop given in the provided files**. The read ports should be all combinational logic. Do not use tri-state logic in your design.

Design this register file such that changing the width to 32-bit or 64-bit is straightforward (e.g., by using parameters).

The read and write data ports are 16 bits each. The select inputs (read and write) are 3 bits each. When the write enable is asserted (high) the selected register will be written with the data from the write port. The write occurs on the next rising clock edge; write data cannot flow through to a read port during the same cycle.

There is no read enable; data from the selected registers will always appear on to the corresponding read ports.

The reset signal is synchronous and when asserted (active high), resets all the register values to 0. The err output should be set to 1 if any register (or other sub-module) in the register file has an error. For now, you may assume that an error only happens in a register if the input or enable is an unknown value, and in other sub-modules if the input is an unknown value. Otherwise, it should be set to 0.

You may find Figures B.8.7 and B.8.8 in your textbook good places to start, when thinking about how to design your register file.

You must use a hierarchical design. Design a 16-bit register first, and then put 8 of them together with additional logic to build the register file.

Do not make any changes to the provided `regFile_hier.v` file.

Problem 2 (30 points)

In Verilog, create a register file that includes internal bypassing so that results written in one cycle can be read during the same cycle. Do this by writing an outer "wrapper" module that instantiates your existing (unchanged) register file module; your new module will just add the bypass logic. The list of inputs and outputs of the outer module should be the same as that of the inner module.

Do not make any changes to the provided `regFile_bypass_hier.v` file.

Problem 3 (10 points)

In this problem, you will gain some experience with Synthesis. Follow the steps outlined in the `synth-howto.pdf` document and get setup. Synthesize your new register file (in the same `hw3_2` directory). Synthesis will create a `synth` folder which will include `rf_bypass.syn.v`, `area report`, `timing report`, etc. Do not delete this directory - it must be included in your submission. Make sure that:

- NO cells in the `synth/cell_report.txt` has zero area in it. If you see a zero area for a module name, then that means that module had some kind of Verilog coding error and did NOT get synthesized. You must fix this problem. Look in `synth.log`, search for that module name, and look for warnings/errors.
- NO cells in the `synth/reference_report.txt` has zero total area. If it does, then some sub modules underneath have failed to synthesize. Check `cell_report.txt`.

Copy the `synth` folder back to your laptop:

```
prompt> cd assignments/hw03/hw3_2
prompt>  scp -rp <your Okta ID>@coe-ee-cad15.sjsuad.sjsu.edu:code/hw03/hw3_2/synth .
```

Testing Instructions

To run the testbench programs that we have distributed for this lab, use:

- `./run test <homework> [problem]`
- Replace <homework> with the assignment name (e.g., hw3).
- The [problem] argument is optional. If omitted, all tests for the selected homework assignment will be run.
- For more detailed output, add -v (e.g., `./run test -v hw3 hw3_1`).
  - **Warning:** -v output can be extremely long in some contexts.

The testbench for this problem (tb_regFile_bypass_hier.v) generates a random set of input signals to your module in each cycle, and compares outputs from your module with outputs that are expected from a perfect register file bypass implementation.

If there are no errors in your design, you will see a *"TEST PASSED"* message. If the testbench failed with a *"TEST FAILED WITH xx ERRORS"* message, look for error messages like *"ERRORCHECK: Read data incorrect in cycle <cycle_number>"* in the testbench output. Above each of these error messages you will see the inputs to your module, your outputs and the expected outputs for that cycle which can help you debug. As in problem 1, we may run additional tests when grading.

Verilog Rules

Read the Verilog rules document at assignments/tools/verilog_rules/verilog_rules.pdf and adhere to the conventions. Rules adherence will be checked when you run `./run test`.

Submission Instructions

In addition to the Verilog, you should also turn in schematics for each of your components. The schematics may be handwritten or computer generated but must be legible if they are handwritten. The schematic files should be named schematic.pdf and placed in the corresponding problems subdirectory (e.g., hw3_1/schematic.pdf). **Any solution without a corresponding schematic drawing will not be graded**. Although the schematics may seem simple for some of these components, as your project gets bigger and bigger, you'll find that drawing schematics of each component and the bigger picture will make your task much, much easier.

To build a submission for grading, run:

- `./run build_submission <assignment>`
- This will automatically run `./run test <assignment>`, then run the generated submission through the integrated autograder.
- A submission ZIP is created in generated_turnins/<assignment>/.

- If you run `build_submission` multiple times for the same assignment, the generated zip files will be prepended with a number. The most recent submission should be the item with the highest number.
- **IMPORTANT:** Running `./run build_submission <assignment>` **DOES NOT** actively submit your assignment, it only generates the file you will submit. This file must be manually uploaded to Gradescope.